

SandBox 入门指北

作者：

- starwalking

i 提示

注意，有的题目可能对于Windows侧不是那么友好，在不用pwntools等工具的情况下，你可能需要使用WSL进行解题，WSL的配置请自行搜索，此处不多赘述

Welcome To SandBox World!

SandBox，中文常译作"沙箱"，是一种把程序关进"笼子"里运行的机制——让程序以为自己拥有完整的世界，实际上能摸到的边界早已被划定好。系统层面（比如内核用 `seccomp` 限制系统调用）有沙箱，语言层面同样有沙箱：限制你能用的关键字、清空你能调用的内置函数，把一段代码的能力收缩到极小的范围里。

当我动笔写这篇入门指北的时候，其实犯了难——沙箱这个话题不太容易找到一条主线讲清楚。不过没关系，本文不贪多，专注讲清楚**语言层沙箱**这一块，也就是大家在 CTF 里更常遇到的形态：以 Python 的 `pyjail` 为代表。系统层的沙箱（`seccomp`、`namespace` 这些）和它关系紧密，但更多归属于 Pwn 方向的知识体系，这里就不展开了，感兴趣的话可以去找 Pwn 方向资料深入学习。

语言层沙箱和安全方向的关系非常紧密，几乎是 Misc/Web 方向绕不开的一环。出题人把一个语言的执行环境"剪秃"，只留下极少的字符、极少的函数，你要做的就是在这个被剪秃的世界里，重新拼凑出执行任意代码的能力。这个过程逼着你去深挖一门语言平时根本不会注意到的角角落落——某种意义上，玩沙箱比单纯背语法更能让你理解一门语言。

接下来，正文部分就是一篇完整的 `Pyjail` 入门——你会看到python沙箱长什么样、怎么搭本地环境练手，以及几类最基础也最常用的"越狱手段"。

什么是语言层 SandBox?

简单来说，语言层沙箱就是给一段代码的执行环境划一个能力边界：哪些关键字不出现、哪些内置函数被清空、代码长度有没有限制，统统由沙箱说了算。和系统层沙箱（依赖内核机制限制系统调用）不同，语言层沙箱通常直接作用在代码或语法层面——过滤字符串、清空 `__builtins__`、限制 AST 节点等等，思路更贴近"语言本身怎么运作"，而不是"操作系统怎么管进程"。

📌 小贴士

常见的语言层沙箱长什么样？

Python 的 `pyjail` 是最经典、资料也最多的一类，本文接下来会重点展开。除此之外还有不少"兄弟款"：

- `jsjail`：限制 JS 运行环境里能用的全局对象和关键字，常见于 Node.js 沙箱题或浏览器端的 CSP 绕过类题目，思路上和 `pyjail` 很像——都是顺着原型链/对象关系找回被收走的能力。
- `bashjail`：限制 shell 命令里能用的字符或命令，典型场景是某些 Web 题给你一个执行命令的接口，但过滤了空格、斜杠之类的关键字符，需要靠通配符、变量拼接、内部命令等手段绕过。
- `Cjail`：偏向于 Pwn 方向，通过裁剪可用的库函数或加上 `seccomp` 限制系统调用来收紧 C 程序的能力边界，玩法上更偏底层，往往需要懂一些汇编和系统调用的知识。

这些沙箱本质上都是同一类思路：在被收紧的语法/函数集合里，找到出题人没堵死的那条路，重新拿回执行能力。本文以 `pyjail` 为主线展开，是因为它资料最完善、上手也最友好；如果你想横向拓展，可以在掌握 `pyjail` 的思维方式后，再去找 `jsjail`、`bashjail` 相关的 writeup 练手——CTF 资源站（比如 CTF Wiki）和各类比赛的 writeup 仓库里都能搜到不少例题，对照着别人的解题过程学起来会比单纯啃文档快很多。

Pyjail 是什么

~~或者说，这一节才是本文真正的主菜~~

pyjail是python jail的简称。简单来说，你需要面对的是一个"被关在笼子"的python环境，只能执行严格受限制的代码。代码长度受到限制，一些字符不让出现，内置函数被清空，诸如此类。(◦-.-◦)

(等等！——连一个内置函数都不给我，print也没有？🙄)

你的目的就是，想办法绕过这个限制，获取执行任意代码的能力，逃离这个"监狱"，最终获取远程靶机的shell权限(权限不高也没关系)

虽然看着有些许困难，但深入了解后，你会发现python原来有那么多"黑魔法"。🤖

Pyjail难不难？

- 对Python新生来说，是非常具有挑战性的题目。**先补Python基础**，Pyjail的题通常会涉及到一些**魔术方法**之类的技巧，因此熟悉Python的语法是必要的，但一点基础也没有也不用担心，本次比赛提供了Python入门的题目，你可以先去了解，跟着题目学习，在解题的时候取逐渐理解Python。
- 对有一定Python经验的选手，每一个Pyjail题目，就像是解一个Puzzle，与出题人斗智斗勇说是。🤖
- 对大佬：我起了，一payload秒了，有什么好说的？。🙄

值得注意的是，Pyjail**难度跨越很大**——有的题目一行秒杀，有的需要精心构造利用链，组合各种绕过手段，更有的需要关注Python版本，甚至Cpython底层机制。🤖

好在，基础的**Payload**还是比较好懂的，只要你看懂下面一行代码，就成功步入Pyjail的世界了(笑\~

```
__import__('os').system('whoami')
```

可以在自己本地尝试一下 Ciallo\~(∠ · ω <)\~☆

如何入门Pyjail？

1. **先学好Python基础**
2. 知道什么是**类**，**实例**，**继承关系**，以及那些带下划线的**魔术方法**，比如 `__class__`，`__subclasses__` 等等
3. 会看报错信息！很多时候报错信息能提示你的代码哪里出了问题，这是非常重要的线索哦 (*^ω^*)

2. 搭建本地实验环境

- 和pwn，web一样，有的时候你不需要直接就去怼靶机。把题目代码复制到本地，自己在本地玩一玩。毕竟有的题目print都不给你，你自己本地那就想怎么print就怎么print了。本地都通不过，远程更没戏嘛\~

3. 学会用 nc 连接远程靶机

- 本地调通之后，最终还是要去远程靶机上把 flag 抠出来。Pyjail 题目大多以 `nc <host> <port>` 的形式给出连接方式 `nc`（netcat）就是用来和远程服务建立一个简单的 TCP 连接的小工具，连上之后你就能像在本地交互一样，把 payload 输进去拿到回显。Linux/WSL 下一般自带 `nc`，Windows侧在本次入门题目中会教怎么用nc连接环境，请移步[从此开始-nc篇](#)，连接形式一般如下：

```
moeops@foo:~$ nc 127.0.0.1 9999
```

- 连上之后，远程那头跑的就是被沙箱关起来的 Python 解释器，你输入的每一行都会被当场拿去执行（或者被拦下来）。如果题目限制比较死，手动一行一行试效率太低，后续可以考虑用 `pwnertools`（Python 下常见的 CTF 交互库）写一个简单的脚本自动收发数据，会顺手很多。

4. 跟着CTF真题练！

- 推荐从一些简单题开始，比如pwn.college的Hanto Dojo提供了一些[入门题目](#)，更多的Pyjail题目需要你们自己去搜索啦，pyjail相对其他类型的题目而言还是比较零散的。

5. 学会查阅Python官方文档！

- 有相当一些题目考察的就是Python某个冷门特性，或者版本更新后的小变化。这时候翻阅[官方文档](#)就是最好的办法啦，不仅可以解出题，还能加深你对Python的理解哦。🤖

理解Pyjail题目的基础"越狱手段"！

- 字符过滤了怎么办？

以下介绍一些手段：

1. 拼接

如果是ban了一个单词，比如'os'，我们可以把'os'拆成'o' 和 's'，'o' + 's'就是os，就不会被检测到了。

1. 函数

Python中有一个内置函数chr，他能把一个整数转换成对应的字符，比如'o' 的ascii码是111，那么 chr(111) 的结果就是'o'，这招同样可以用于数字字符被ban，比如通过ord('o')获取数字111。

1. 编码

有的时候，chr不让你用怎么办，那就编码，Python的字符是支持16进制，8进制的，比如111的16进制是6f，那么'\x6f'就是'o'，同理，8进制'\157'也是'o'，可以自己尝试print一下)

4. 规范化

Python3有字符规范化规则，NFKC，比如，在未经特殊处理的情况下，Python会认为 breakpoint() 和 brekapoint() 都是调用函数breakpoint，所以，即使有些场景禁用了却大多数字符，我们依旧可以使用这个特性来进行绕过

还有很多很多绕过手段！就等大家自行探索吧~

- 探测环境

一些Python的内置函数，可以告诉我们当前有什么可用利用的手段，比如：

dir() —— 列出当前作用域的所有名称

locals() / globals() —— 查看局部/全局变量字典

help() —— 查看一些帮助信息

探测环境常见于我们不知道全部源码，或者完全不知道源码的情况下，进行查看。

~~虽然出题人大多时候这些也不给你，纯盲打~~

- 对象关系链

Python里面万物皆对象

所有对象的基类，最终指向object

所有对象的元类，最终指向type

因此，我们可以通过这种继承关系进行逃逸

```
obj.__class__ # 获取对象obj的类
cls.__base__ # 获取类cls的直接父类
cls.__subclasses__() # 获取类cls的所有直接子类
```

比如下面这个payload

```
[].__class__.__base__.__subclasses__()[158].__init__.__globals__['__builtins__']
```

[]是一个对象，它的父类是。

list.__base__，得到。

object.__subclasses__()，获取所有直接子类的列表

取索引158，拿到子类 <class 'os._wrap_close'>

i 提示

注意，索引158不是固定的，不同Python版本中索引可能不同

拿到这个类有什么用呢，它的初始化魔术方法 __init__ 中有一个字典 __builtins__，在清空了内置模块 __builtins__ 的情况下，我们就能成功恢复 __builtins__ 啦。

进一步的，我们就能恢复 exec，input 之类的内置函数，从而成功"越狱"😁。

4. 补充说明

- 对于一些简单题而言，绕过手段通常由多种，所以做出来的不一定是非预期哦。
 - Pyjail禁止手段很多，比如禁用ast节点，审计钩子，限制字节码等等，也通常有绕过的手段，由于篇幅限制，不宜展开说明，此处提及一下，栈帧逃逸，覆盖函数，重载模块，反序列化，感兴趣的同学可以自行查阅🔍。
-

再提几句

其实，Pyjail也不是没有一把梭工具🔪，可以看看Typhon，只不过目前只能做一些相对简单的题目，一把梭工具🔪虽然好，但究其目的，是要提升自己的技术，所以，不要太依赖了hh。

因此，希望各位同学，在做题过程中，也不要太利用AI手段，AI究其只是一种辅助，~~纯依赖AI的话，那么需要技术员(你)做什么呢，回车吗。~~

XDSEC的各位都欢迎大家来锤🔨，笔者出的部分题可能有点新生不友好，但深入理解后，相信你一定能做出来的~~(其实纯折磨新生(bushi。~~

衷心希望大家能在本次比赛中了解CTF的一些知识，提升自己的技术。

致谢

感谢你能成功阅读到这里，Pyjail可能会劝退一些同学，但请你不要放弃，技术的提升不是一朝一夕的，相信你能在做Pyjail的过程中更深入了解Python的机制。

笔者水平有限，或许不能带领新同学真正感受到Pyjail的魅力，如有错误望指正。